

26  
Jul  
26

CSA1407 - Computer Design  
ASSESSMENT TOOL 2

K. Balaji  
192624255

① a) Error Identification

b) Phase Classification.

Line 11: Syntax error - missing semicolon; after int arr[5]

Line 13: Semantic error - call to sum(arr) pass 1, 2.

Line 15: Syntax error - missing semicolon; after printf("%").

Line 5: logical error - off by one error in loop condition  $i < n$ .

Line 14: Runtime error - 1 by zero

Line 20: Runtime error - uninitialized pointer dereference.

c) Compilation Behaviour

→ compile time errors: line 11 and 15 and line 13 are caught during compilation

→ Runtime error: line 5, 14, and 20 pass compilation

→ cascading effect: Syntax lexical error halt parsing.

d) Code Correction

```
#include <stdio.h>
```

```
int sumA (int arr[], int n) {
```

```
    int i, sum = 0;
```

```
    for (int i = 0; i < n; i++) { sum = sum + arr[i]; }
```

```
    return sum;
```

```
}
```

```
int main () {
```

```
    int arr[] = {1, 2, 3, 4, 5};
```

```
    int total;
```

```
    total = sumA(arr, 5);
```

```
    if (total == 15) {
```

```
        printf("Correct sum\n"); }
```

```
Print a=10, b=2;
```

```
Print c=a/b;
```

```
Print val;
```

```
Print *p = &val;
```

```
*p=20;
```

```
return 0; }
```

### c) Impact Analysis:

- Division by zero: Triggers an arithmetic CPU exception, causing immediate OS processing termination
- uninitialized pointer: writing to an unmapped pointer causes a segmentation fault or silently corrupts code
- Out of bounds access: → can overwrite adjacent memory locations, causing unpredictable data corruption or process crashes.

### (f) Advanced Compiler Perspective

- ↳ Static Analysis Integration: Perform compile time data flow analysis to flag uninitialized variables.

### Enhancements for Debugging

- Enhanced diagnostic warnings (^)
- Automated Error Recovery (insert missing tokens)

## QUESTION 2

### Lexical Error Identification, classification & token correction

| Line | invalid token | Reason                             | corrected token |
|------|---------------|------------------------------------|-----------------|
| 3    | 1 num         | cannot begin with numeric          | num1            |
| 4    | rate@         | contains illegal special character | rate            |
| 5    | 'abc'         | character literal                  | "abc"           |
| 6    | 50\$ 20       | illegal symbol \$                  | 50 + 20         |
| 7    | num#1         | # is illegal in id                 | num_1           |
| 8    | %count        | cannot start with operator         | count           |
| 9    | "welcome?..   | unterminated string                | "welcome..."    |
| 10   | 09            | invalid octal prefix               | 9               |
| 11   | 0x212         | invalid hex literal                | 0xA12           |
| 12   | 12.3.4        | constant formation                 | 12.34           |
| 13   | "Hello        | invalid termination                | "Hello"         |
| 14   | Invalid id    | Hyphen is treated as minus         | Invalid id      |
| 15   | 6..6          | invalid float format               | 5.6             |
| 16   | 10@5          | @ is unrecognized                  | 10 * 5          |
| 17   | 'xy'          | character literal                  | 'x'             |
| 18   | 3value        | cannot start with numeric          | value 3         |

### d) Compiler Behavior Analysis

Lexer operation: The lexical analyzer uses Regular expressions and finite automata to tokenize input character by character.

Error Recovery: Modern compilers do not stop immediately

## 2) Advanced Thinking

→ DFA corrupts for invalid digit leading identifiers.

↳ Start state : Reads a digit [0-9] → state  $S_1$

↳ State  $S_1$  : [a-zA-Z] →  $S_{next}$ .

→ Beginner error hints : convert cryptic messages to context aware hints.

## Question 3

### Error Identification & phase classification

Ans → 1. Syntax error ( ; missing )

↓ 8. Lexical error ( # X )

9. Syntax error : missing semicolon ;

11. Semantic error : Argument mismatch (dim1 (R))

12. Semantic issue : using = instead ==

13. Syntax error : missing ;

16. Runtime error : div by 0 ;

18. Runtime error : Division by zero (x/0)

20. Runtime error : Dereferencing ptr

22. Optimization issue +0,

23. Logical error - infinite loop.

### ( ) Compilation vs Execution

Compile time errors : lines 9, 11, 12, 13 are flagged during

Compilation because they violate language grammar.

Execution error : line 16, 18, 20, 23 pass compilation and execute resulting crash signals.

(D) Fully Connected Program,

(C) #include <stdio.h>

```
int divide (int a, int b) {  
    return (b != 0) ? (a/b) : 0;  
}
```

```
int main () {
```

```
    int value = 20;
```

```
    float num = 15.5;
```

```
    int result;
```

```
    result = divide (10, 2);
```

```
    if (result == 5) {
```

```
        printf ("Result is %d\n", result);  
    }
```

```
    int x = 10;
```

```
    int y = 2;
```

```
    int z = x / y;
```

```
    int arr[4] = {1, 2, 3, 4};
```

```
    printf ("y.d\n", arr[3]);
```

```
    int dummy = 25;
```

```
    int *ptr = &dummy;
```

```
    *ptr = 25;
```

```
    int total = z + y * result;
```

```
    while (x < 20) {
```

```
        printf ("-y.d\n", x);
```

```
        x++;  
    }
```

```
    return 0;
```

```
}
```

## (E) Deep Analysis

→ Increased Pointer Dereference:

Dereferencing an uninitialized or wild pointer writes data to arbitrary memory addresses.

→ Array Bounds Validation: Accessing `arr[4]` reads or writes beyond allocated stack space.

→ Operator Precedence in Intermediate Code: Improved evaluation ordering in intermediate representation computes expressions.

## (F) Advanced Task:

1) Early Runtime Error detection: use static Analyzer and abstract interpretation during compilation to detect value ranges.

2) Improved Debug messages: include source code snippets directly in error reports along with "Fix hints".

3) Optimization via constant folding and Algebraic Simplification.